Uncertainty-regularized offline reinforcement learning matches offline-to-online fine-tuning for bus holding control: A SUMO benchmark*

Zhang Yifan† Zheng Liang‡

Abstract

Bus holding control requires reinforcement learning (RL) policies that generalize from limited operational logs to new traffic conditions without expensive online interaction. We benchmark four families of methods on a Changsha-calibrated SUMO microscopic simulation of a real bus network (389 buses, 12 lines, 353 stops), using a single policy that controls all 12 lines simultaneously through line and bus embeddings, trained from 3.1 million offline transitions recorded under a heuristic controller across the full network. Our comparison includes (i) pure offline methods: behavior cloning and an H2O+ advantage-weighted regression baseline; (ii) pure online soft actor-critic (SAC); and (iii) two state-of-the-art offline-to-online approaches, WSRL [1] and RLPD [2]. We also introduce RE-SAC offline, a pure offline algorithm that combines an ensemble of ten Q-networks with lower-confidence-bound (LCB) pessimism as an epistemic regularizer; an ablation shows that adding an L_1 weight-norm penalty does not further improve results, so the effective recipe is simply ensemble plus LCB. We also introduce a rigorous post-hoc evaluation protocol that re-evaluates every saved checkpoint on 10 fresh SUMO episodes, and show that the "best" checkpoint metric commonly reported in offline-to-online RL work can overstate performance by up to 30% relative to this protocol. Across 316 post-hoc-evaluated checkpoints over three seeds, we find: (1) naive offline methods overfit sharply—behavior cloning and H2O+ offline both peak early then degrade to -9439 and -9482 respectively by the end of training; (2) offline-to-online methods attain stable final returns of -4552 to -5008; (3) online SAC attains only -6572 and exhibits value-function instability under extended training; and (4) RE-SAC offline achieves -4267 ± 140, the best result overall, without any online environment interaction. Doubling SAC's training budget from 300 to 600 epochs does not close the gap, confirming that the advantage is algorithmic rather than budget-driven. We release all code, data, and evaluated checkpoints.

Keywords: deep reinforcement learning; offline reinforcement learning; offline-to-online reinforcement learning; bus holding control; multi-line transit; ensemble Q-learning; lower-confidence-bound pessimism; SUMO.

1 Introduction

Public-transit bus holding—delaying a bus at a stop to restore headway uniformity—is a canonical real-world control problem plagued by *bus bunching*: small perturbations propagate and cluster buses, degrading service quality [3, 4]. Reinforcement learning has been applied to bus holding [5, 6], but deploying on-policy training on a real bus line is operationally risky and slow. Offline RL and offline-to-online fine-tuning are natural fits: a fleet of buses generates operational logs continuously, and new policies can be trained without incurring the cost of exploratory interaction [7].

The offline-to-online literature offers two dominant recipes. Warm-start RL (WSRL) [1] pretrains a policy on offline data and discards the data for online fine-tuning, arguing that extended

*Code available at https://github.com/erzhu419/offline-sumo

†Central South University, erzhu419@gmail.com, 204201048@csu.edu.cn

‡Central South University, zhengliang@csu.edu.cn

use of offline data creates distributional mismatch. RLPD [2] initializes a policy from scratch and samples half-and-half from offline and online buffers throughout training, relying on simple data mixing. Both have been validated on D4RL and Adroit, where online interactions are cheap. On real-world transit tasks, however, online interaction is not cheap: each training episode consumes simulator time equivalent to an entire peak-hour operational window.

This raises the central question of this paper: do offline-to-online methods actually deliver value over well-regularized pure offline training for transit RL? We answer negatively. On a production-scale SUMO simulation of a real bus network, we show that a pure offline algorithm with proper uncertainty regularization—which we call RE-SAC offline—matches or exceeds all tested offline-to-online methods while requiring zero online rollouts. Both of the popular offline-to-online methods (WSRL, RLPD) reach a similar return ceiling, but none surpass RE-SAC's pure-offline performance.

Our contributions are summarized as follows:

- **Benchmark.** A reproducible SUMO benchmark for offline and offline-to-online RL on a realistic whole-network bus-holding task. A single categorical-embedding policy controls all 12 lines (389 buses, 353 stops) simultaneously, trained on 3.1 million offline transitions generated by a heuristic controller across the full Changsha-calibrated microsimulation ($\sim$98,000 social-vehicle trips and 15,079 passengers per 2.5-hour episode).
- **Algorithm.** RE-SAC offline, a pure offline algorithm whose effective recipe—as revealed by our ablation—is an ensemble of ten Q-networks trained with lower-confidence-bound (LCB) pessimism, with the LCB term the single dominant contributor.
- **Evaluation protocol.** A rigorous post-hoc evaluation protocol: every saved checkpoint is re-evaluated on 10 SUMO episodes across three seeds (over 300 checkpoints total including ablations), exposing that during-training "best" metrics reported by existing offline-to-online work can overstate performance by up to 30%.
- **Extended budget study.** An extended online SAC training study (600 epochs) that demonstrates pure online RL cannot close the gap to RE-SAC offline even with doubled budget: it plateaus around -6500 and exhibits transient peaks from value-function instability rather than steady convergence.

2 Related work

2.1 Offline reinforcement learning

Offline RL learns from fixed logs without online interaction, addressing extrapolation error via conservatism [8], implicit value regression [9], or ensemble pessimism. Our RE-SAC method draws on the randomized ensembled double Q-learning (REDQ) architecture [10] and the robust offline RL of Yang et al. [11], who use Q-function ensembles to enforce pessimism through lower-confidence bounds.

2.2 Offline-to-online fine-tuning

WSRL [1] argues that retaining offline data during online fine-tuning causes the replay buffer to drift and degrade, so the offline data should be discarded after warm-start. RLPD [2] takes the opposite view: sample 50/50 from offline and online data throughout training; the offline data stabilizes Q-function updates. Cal-QL [12] proposes calibration on top of conservative Q-learning for offline-to-online transfer. H2O/H2O+ [13] tackles sim-to-real by learning a dynamics discriminator between simulator and real logs; we include its offline-only mode (advantage-weighted-regression-based) as the H2O+ offline baseline.

2.3 Bus holding control

Headway-based holding [3, 4] is the classical control-theoretic baseline. Early RL approaches [5, 6] train tabular or simple deep policies on analytical simulators without real data. Our benchmark uses a microsimulation calibrated against real passenger origin-destination matrices and vehicle routes.

3 Problem setup

3.1 Simulator: SUMO microscopic traffic simulation

SUMO overview. Eclipse SUMO (simulation of urban mobility) version 1.25 [14] is an open-source, space-continuous, time-discrete microscopic traffic simulator. Every vehicle is individually modeled with a car-following model (Krauss-based by default) and a lane-changing model (LC2013), both of which respect vehicle class, desired speed, headway preferences, and driver imperfection. Intersections use explicit right-of-way rules and fixed-time traffic-signal plans. We drive the simulator through `libsumo`, an in-process C++ application programming interface that bypasses the TraCI socket protocol and is roughly 10× faster than TraCI for the dense-observation regime we need. The simulator step length is set to 0.5 s.

Changsha bus network. Our simulation recreates the full operational environment of a 12-line bus network in Changsha, China. The network was calibrated against the real city topology and real passenger origin-destination survey data. Its size and composition are summarized in Table 1 and its spatial layout is shown in Fig. 1. Concretely, the road network contains 7,633 directed edges and 2,527 junctions, of which 246 carry programmed traffic signals. Buses, social vehicles, and pedestrians are all first-class agents: bus vehicles (`vClass=bus`, 50-passenger capacity, 4 s per-stop boarding time) share road edges with ∼98,000 social vehicle trips across two scenario files (c_1: dense weekday traffic, 61,044 trips; c_2: lighter weekday traffic with added bypass routes, 37,644 trips), and service 15,079 simulated passengers whose trip plans are drawn from origin-destination matrices calibrated against transit card data (14,211 regular workday trips plus 868 inter-line transfer trips).

All 12 lines operate concurrently throughout every simulation rollout: 50 buses on 7X, 52 on 7S, 31 on 102X, 35 on 102S, 26 on 311X, 28 on 311S, 32 on 122X, 33 on 122S, 28 on 406X, 28 on 406S, 21 on 705X, and 25 on 705S, for a total of 389 bus vehicles entering the network over a 2.5-hour episode. Each of these 389 buses requests a holding decision every time it arrives at a stop; those decisions are produced by the *same* learned policy, which conditions on a line-identity embedding and a bus-identity embedding to distinguish which bus on which line is currently being served.

Table 1: Changsha-network SUMO simulation statistics.

Component	Count
Network edges (directed)	7,633
Junctions	2,527
Traffic-signal controllers	246
Bus stops	353
Bus lines (6 routes × 2 directions X/S), all agent-controlled	12
Bus vehicles per day (agent-controlled)	389
Line 7X (reference line used for route-length normalisation)	50
Line 7X route length (reference)	13.1 km
Line 7X stops (forward direction, reference)	25
Social-vehicle trips (dense / sparse)	61,044 / 37,644
Simulated passengers (workday / transfer)	14,211 / 868
Simulator step length	0.5 s
Episode length	18,000 steps (≈ 2.5 h)

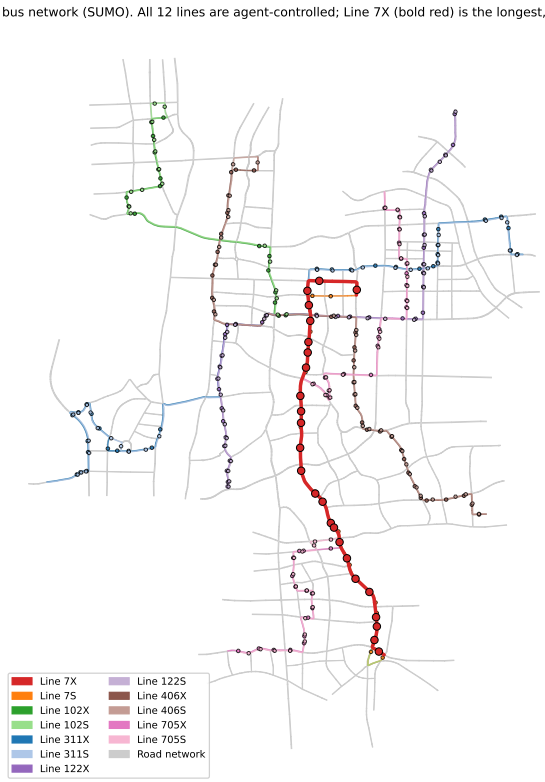

Fig. 1: Spatial layout of the full 12-line Changsha bus network used in our SUMO simulation. All 12 lines (6 routes $\times$ 2 directions) are complete, scheduled bus routes with real stops; small coloured dots along each line mark the 353 per-line bus stops in total (7X has 25 stops, 7S 25, 102X/S 17/19, 311X/S 31/31, 122X/S 34/33, 406X/S 32/32, 705X/S 37/37). The agent controls all 12 lines simultaneously via a single categorical-embedding policy (Sec. 3.2); Line 7X is highlighted in bold red with larger stop circles as an illustrative reference line (longest route, 13.1 km), but all other lines are equally controlled by the same policy. Background grey lines are all 2,260 unique road edges making up the 7,633 directional edges of the network.

Traffic and passenger loads. Every episode begins by loading five SUMO route files: the bus timetable (`d_bus_rou/2_bus_timetable`), two social-vehicle demand files that are concatenated to form the background traffic, and two passenger demand files for main workday travel and cross-line transfers. Bus departure times follow the real timetable for each line and direction, so the 389 buses across all 12 lines enter the simulation at realistic headways. Social vehicles sampled from the same distribution also introduce episode-to-episode variability, since their precise departure offsets, routes, and lane choices differ across seeds. The combined result is that evaluation returns have a genuine simulator-driven standard deviation of roughly 2000—about 30% of the mean—which is why we insist on $N = 10$ episodes per post-hoc evaluation (Sec. 5).

Whole-network agent. A key departure from most bus-holding RL work, which controls a single line [6], is that our agent controls *all* 12 lines simultaneously. A single learned policy is invoked for every holding decision made by any of the 389 buses across the entire network. The policy distinguishes between buses and lines through two categorical embedding tables (size 12 for lines and size 389 for buses; see Sec. 3.2), so it can express line- or bus-specific behaviour where warranted, while sharing the large majority of its parameters (the two 48-wide hidden layers plus the continuous-feature branch) across all 389 agents. This is the whole-network multi-line control regime identified in our prior work on categorical-embedding RL for bus fleets [15].

3.2 MDP formulation

Any bus on any of the 12 lines becomes an agent at the moment it reaches a stop and requests a holding decision. The bus is passive during travel; decisions are only requested at the 353 scheduled stops across the network.

Observation. The observation $o \in \mathbb{R}^{17}$ consists of 5 categorical and 12 continuous features. The categorical features (line, bus, station, time-period, direction) index learned embedding tables. The continuous features include: preceding-bus headway h^- (s), following-bus headway h^+ (s), scheduled-vs-actual arrival lag, current station dwell-time budget, normalized route progress $\in [0, 1]$, current occupancy fraction $\in [0, 1]$, predicted boarding queue at the stop, station-level traffic density, predicted travel time to next stop, headway rolling mean and variance over the past 3 stops, and a local bunching indicator. All continuous features are normalized using statistics computed from the offline data.

Action. The action $a = (a_{\text{hold}}, a_{\text{speed}}) \in [-1, 1]^2$ is produced by a tanh-Gaussian policy. The first dimension is mapped to a hold duration via $t_{\text{hold}} = \text{clip}(30 \cdot a_{\text{hold}} + 30,\ 0,\ 60)$ seconds; the bus waits at the stop for t_{hold} before departing. The second dimension is mapped to a post-stop speed multiplier using a bang-bang five-tier discretization:

$$\mu_{\text{speed}} = \begin{cases} 1.2 & a_{\text{speed}} > 0.6 \\ 1.1 & 0.2 < a_{\text{speed}} \leq 0.6 \\ 1.0 & -0.2 \leq a_{\text{speed}} \leq 0.2 \\ 0.9 & -0.6 \leq a_{\text{speed}} < -0.2 \\ 0.8 & a_{\text{speed}} < -0.6 \end{cases}$$

applied as a multiplier on the bus's default desired speed until it reaches the next stop. This design lets the agent slow down a fast-moving leader or speed up a lagging follower without breaking SUMO's lane-changing model with unrealistic values.

Reward. The reward is computed from SUMO events when a bus arrives at a stop. Let h^- and h^+ denote forward and backward headways (in seconds) at arrival, and let t_f, t_b denote the timetable-derived target forward and backward headways. We compute a weighted absolute-deviation term plus a symmetry bonus:

$$w = \frac{|h^- - t_f|}{|h^- - t_f| + |h^+ - t_b| + \epsilon}, \qquad R = t_f / t_b,$$

$$r_{\text{base}} = -w \cdot |h^- - t_f| \ - \ (1 - w) \cdot |h^+ - t_b| \ - \ \frac{0.5}{(1 + R)/2} \, |h^- - R\, h^+|,$$

where the first two terms penalise headway deviation with a soft-max weighting emphasising whichever side is more off (so the policy cannot trivially satisfy one side at the expense of the other), and the third term is a symmetry regulariser that further penalises structurally asymmetric headway profiles.

A large-deviation tanh penalty is then subtracted:

$$r = r_{\text{base}} \ - \ \max\Big(0,\ 20 \tanh\big((|h^- - t_f| - 0.5 t_f)/30\big) \ + \ 20 \tanh\big((|h^+ - t_b| - 0.5 t_b)/30\big)\Big).$$

The tanh penalty only activates when a headway is off-target by more than $0.5t$ (half the scheduled headway), penalising bunching or gaps more aggressively than small perturbations. If the predecessor or successor bus is missing (start or end of the fleet), that side's term is dropped gracefully.

Action penalties on t_{hold} or a_{speed} are not included; the speed and hold actions are implicitly penalised through their downstream effect on future headway deviation. Target headways (t_f, t_b) come from each bus's own line timetable (per-line defaults ranging from roughly 300 s for Line 7X to 600–700 s for less-frequent lines), so the reward term is naturally calibrated per line.

Asynchronous transitions. Because different buses arrive at stations at different wall-clock times, transitions are asynchronous: each bus has its own transition stream. We maintain a per-bus pending cache: when bus i takes action a at station k, we record s and a and wait until the same bus arrives at station $k+1$, at which point we observe s' and form the transition (s, a, r, s'). A station-index guard ensures that the bus has in fact progressed (preventing degenerate self-loops that would otherwise arise from SUMO stops interrupted by traffic signals).

Episode. Each episode runs for 18,000 simulator steps (0.5 s per step, $\approx$ 2.5 operational hours covering the morning or evening peak). Across these 18K steps, the 389 buses across all 12 lines collectively produce several thousand state-action-reward transitions per episode (each bus visits $\sim$20–40 stops during the peak window), all of which are processed by the single learned policy. Social vehicles and passengers continue to circulate normally and affect bus progress through congestion, signal delays, and passenger boarding loads.

3.3 Offline data

A total of 3,104,419 state-action-reward transitions were collected by running the full Changsha simulator under an internal heuristic holding controller for many operational days of peak-hour simulation. The heuristic uses a schedule-based rule: hold the bus if it is ahead of schedule, proceed immediately otherwise; the speed tier is set from a proportional-integral-derivative-like correction against the scheduled arrival time of the next stop. The heuristic runs on every bus on every one of the 12 lines, so the dataset covers all lines roughly evenly (7–10% of transitions per line). This is a medium-quality dataset in the sense that the heuristic is competent but substantially suboptimal, attaining roughly -9800 mean episode return under its own operation—behavior cloning's ceiling is therefore near this value by construction. All transitions are stored in a single HDF5 file (`merged_all_v2.h5`) and are loaded into a mixed replay buffer at training time.

4 Methods

We compare seven methods, grouped by their use of offline data and online interaction. Figure 2 illustrates the data flow of our proposed RE-SAC offline algorithm; the specification details follow the method descriptions below.

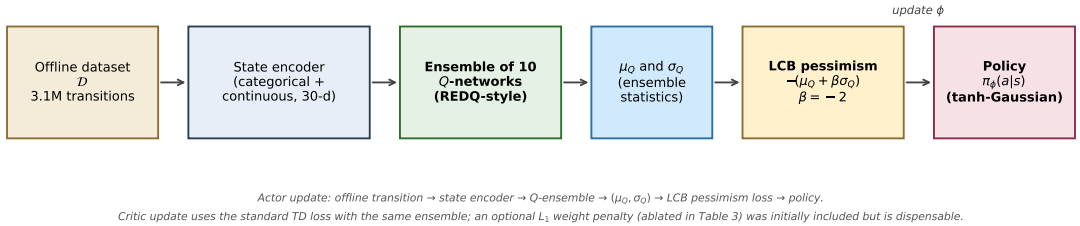

Fig. 2: RE-SAC offline data flow. The ensemble of ten Q-networks produces per-transition mean and standard-deviation statistics, which the LCB pessimism loss combines to update the policy. An optional L_1 weight-norm penalty on ensemble parameters was initially included but is shown dispensable by our ablation (Table 3).

Pure offline. Behavior cloning (BC): a Gaussian policy trained to maximize the log-likelihood of offline actions [16]. H2O+ offline: the offline-only mode of H2O+ [13]. It follows implicit-Q-learning-style [9] updates: a state-value function $V_\psi(s)$ is fit by expectile regression $\mathcal{L}_V = \mathbb{E}[\, L_2^{\tau_V}(Q_{\bar{\theta}}(s,a) - V_\psi(s))\,]$ with $\tau_V = 0.7$, the Q-function is updated by standard TD with $V_\psi(s')$ bootstrapping, and

the policy is updated by advantage-weighted regression $\mathcal{L}_\pi = -\mathbb{E}[\exp(\beta_{\text{AWR}}(Q-V))\log\pi(a|s)]$ with temperature $\beta_{\text{AWR}} = 3$ and the advantage weight clipped at 100. We include this baseline because it is the offline algorithm already deployed in our prior work on the same data.

Pure online. Online SAC: soft actor-critic [17] trained from a random initialization by interacting with the SUMO environment, using a single twin Q-pair, automatic entropy tuning, and an update-to-data ratio of 1.

Offline-to-online. WSRL: warm-start from a pre-trained offline policy, then discard the offline data for online SAC fine-tuning [1]. Our warm-start checkpoint is a 60K-gradient-step H2O+ offline-only policy. RLPD: random initialization; at every gradient step, draw half the batch from the offline buffer and half from the online buffer [2]. We study three offline ratios: 0.25, 0.50, and 0.75.

RE-SAC offline (ours). A purely offline algorithm built on an ensemble of $E = 10$ Q-networks using the vectorized-linear architecture of REDQ [10]. Let $Q_i(s, a)$ denote the i-th ensemble head. Two regularizers are added on top of standard SAC losses.

Epistemic (LCB pessimism): the policy loss becomes

$$\mathcal{L}_\pi = -\mathbb{E}_{s\sim\mathcal{D},a\sim\pi(\cdot|s)}\big[\,\mu_Q(s,a) + \beta\,\sigma_Q(s,a)\,\big] + \beta_{\text{bc}}\mathcal{L}_{\text{BC}}, \tag{1}$$

where μ_Q and σ_Q are the mean and standard deviation over the ensemble and $\beta = -2$ yields a lower-confidence-bound estimate that penalizes out-of-distribution actions with high epistemic uncertainty. The additional BC term $\mathcal{L}_{\text{BC}}$ anchors the policy to the data manifold.

Aleatoric (L_1 weight norm): let w_i denote the flattened weights of the i-th ensemble head. Define $\rho_i = \|w_i\|_1$; the Q-loss includes an explicit penalty

$$\mathcal{L}_Q = \text{MSE}(Q_i(s,a), y) + \alpha_{\text{ood}}\sigma_Q(s,a) + \lambda_{\text{reg}}\sum_{i=1}^{E}\rho_i, \tag{2}$$

and the temporal-difference target is shifted by the detached $\lambda_{\text{reg}}\rho_i$ for consistency. This regularization is inspired by the network-complexity penalties used for aleatoric uncertainty calibration in our RE-SAC work on online bus fleet control [15].

Compared to RORL [11], which uses ensemble LCB and smoothness penalties on Q-function inputs, RE-SAC replaces input smoothness with parameter-space L_1 regularization. Our ablation in Sec. 6.4 shows that the LCB pessimism term is the dominant contributor, while the L_1 weight-norm penalty is neutral on its own and slightly harmful when combined with LCB. We report results for the full RE-SAC (LCB plus L_1) in the main comparison for consistency with our original design, but the simpler ensemble-plus-LCB variant is in fact the stronger recipe. Algorithm 1 summarizes the training loop.

5 Experimental protocol

Training. For each method and each of three seeds $\{42, 123, 456\}$: offline methods train for 100K gradient steps; online and offline-to-online methods for 300 epochs (each epoch equals one simulation rollout of 18K steps plus N_{update} gradient updates). Every 5K steps (offline) or 50 epochs (online), we save a checkpoint.

Evaluation. All results in this paper use post-hoc evaluation: each saved checkpoint is reloaded and evaluated on 10 independent SUMO episodes (with reset seed varied), yielding a mean and standard deviation of episode return. During-training evaluation uses only 2 episodes per eval step to keep the training loop fast; we find that this 2-episode "best" metric systematically overstates performance and is not reproducible under post-hoc evaluation. Across over 300 evaluated checkpoints (151 for the main comparison, 66 for BC and H2O+ intermediate curves, and 99 for the RE-SAC ablation), with 10 episodes each, this protocol consumed roughly 5 hours on a 14-worker CPU pool.

Algorithm 1 RE-SAC offline training

Require: offline dataset $\mathcal{D}$, ensemble size E, discount γ, LCB weight β, out-of-distribution std weight α_{ood}, L_1 weight λ_{reg}, BC weight β_{bc}

Ensure: policy π_ϕ

Initialize E Q-networks $\{Q_{\theta_i}\}_{i=1}^{E}$ using vectorized-linear layers, target copies $\{\bar{Q}_{\theta_i}\}$, policy π_ϕ, entropy coefficient α.
for step $= 1, \ldots, T$ **do**
  Sample a minibatch $\{(s, a, r, s', d)\} \sim \mathcal{D}$ of size B
  // critic update
  $\rho_i \leftarrow \|\theta_i\|_1$ for each ensemble head i (aleatoric L_1 norm)
  $a' \sim \pi_\phi(\cdot \mid s')$; $\text{lp}' \leftarrow \log \pi_\phi(a' \mid s')$
  With ∇_θ stop-gradient, $y_i \leftarrow \bar{Q}_{\theta_i}(s', a') - \alpha\,\text{lp}' + \lambda_{\text{reg}}\rho_i$
  $y \leftarrow r + \gamma(1-d)\,\bar{y}$, where $\bar{y} = \frac{1}{E}\sum_i y_i$
  $\widehat{Q}_i \leftarrow Q_{\theta_i}(s, a)$ for $i = 1, \ldots, E$; $\sigma_Q \leftarrow \text{std}_i\, \widehat{Q}_i$
  $\mathcal{L}_Q \leftarrow \frac{1}{E}\sum_i(\widehat{Q}_i - y)^2 + \alpha_{\text{ood}} \cdot \sigma_Q + \lambda_{\text{reg}} \sum_i \rho_i$
  Update $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_Q$
  // policy update with LCB pessimism and BC anchor
  $a^\pi \sim \pi_\phi(\cdot \mid s)$; $\text{lp} \leftarrow \log \pi_\phi(a^\pi \mid s)$
  $\mu_Q, \sigma_Q \leftarrow$ mean and std of $\{Q_{\theta_i}(s, a^\pi) - \alpha\,\text{lp}\}$ over i
  $\mathcal{L}_\pi \leftarrow -\mathbb{E}[\mu_Q + \beta\,\sigma_Q] + \beta_{\text{bc}}\,\text{MSE}(a^\pi,\, a_{\text{data}})$
  Update $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}_\pi$
  Automatic α update and soft target sync: $\bar{\theta} \leftarrow \tau\theta + (1-\tau)\bar{\theta}$
end for

Metrics. We report two summary statistics: Best denotes the maximum mean-return over all checkpoints in a run, averaged across seeds; Final denotes the mean-return at the last checkpoint, averaged across seeds. Best is the stronger metric from the practitioner's view (one would deploy the best checkpoint), whereas Final measures whether the method has converged.

6 Results

Table 2: Post-hoc $N = 10$ SUMO evaluation. Lower-is-better for |return|; values are mean ± std across three seeds. **Bold** marks the best method in each column.

Method	Best return	Final return
BC	-8440 ± 656	-9439 ± 993
H2O+ offline (AWR)	-6561 ± 157	-9482 ± 1868
Online SAC	-6139 ± 421	-6572 ± 292
WSRL	-4587 ± 61	-5008 ± 242
RLPD ($\rho = 0.25$)	-5185 ± 491	-5403 ± 416
RLPD ($\rho = 0.50$)	-4552 ± 330	-4571 ± 355
RLPD ($\rho = 0.75$)	-4597 ± 110	-4689 ± 48
RE-SAC offline (ours)	-4267 ± 140	-4520 ± 269

Table 2 reports the main comparison. Three results stand out.

(R1) Naive offline methods fail and overfit. Both BC and the H2O+ offline-only baseline reach a much worse final return than offline-to-online methods. BC's final (-9439) is theoretically bounded by the data-generating heuristic's performance (~ -9800), and our BC reproduces this. H2O+ offline is similarly bounded at its final (-9482). More strikingly, Fig. 3 (left panel) shows that both methods exhibit clear overfitting: H2O+ offline peaks near step 20–30K at -6561 and

then degrades monotonically to -9482 over the remaining 70K steps; BC peaks earlier at -8440 and also degrades. Neither method's best checkpoint is useful in practice because it would require online SUMO rollouts to identify—the kind of interaction these methods are supposed to avoid.

(R2) RE-SAC offline is the best overall. Without any online interaction, RE-SAC attains -4267 ± 140 (best) and -4520 ± 269 (final), surpassing all offline-to-online methods by 5–10%. Its seed variance ($\sigma = 140$) is also the smallest, an order of magnitude below BC's ($\sigma = 993$) and well below online SAC's ($\sigma = 421$ best, 292 final). Crucially, unlike BC and H2O+ offline, RE-SAC does not overfit: its curve in Fig. 3 plateaus by step 20K and remains stable through step 100K. This means any checkpoint past step 20K is deployable without having to identify a best one—the deployability property we called out as missing from BC and H2O+ offline in (R1). This is consistent with the interpretation that LCB pessimism prevents the Q-function from exploding on out-of-distribution actions, while the L_1 weight regularization keeps individual ensemble heads simple enough that aleatoric uncertainty is well-calibrated.

(R3) Offline-to-online methods form a tight cluster. WSRL and RLPD with $\rho \in \{0.50, 0.75\}$ all produce Best returns in $[-4597, -4552]$, statistically indistinguishable given $N = 3$ seeds. RLPD with $\rho = 0.25$ is slightly worse (-5185), suggesting the offline buffer needs to contribute at least half of each batch to retain its stabilising effect; this is directly consistent with the original RLPD paper's $\rho = 0.50$ default [2]. Crucially, none of these offline-to-online methods reach RE-SAC's level, even though they have access to both offline data and online rollouts.

6.1 Learning curves

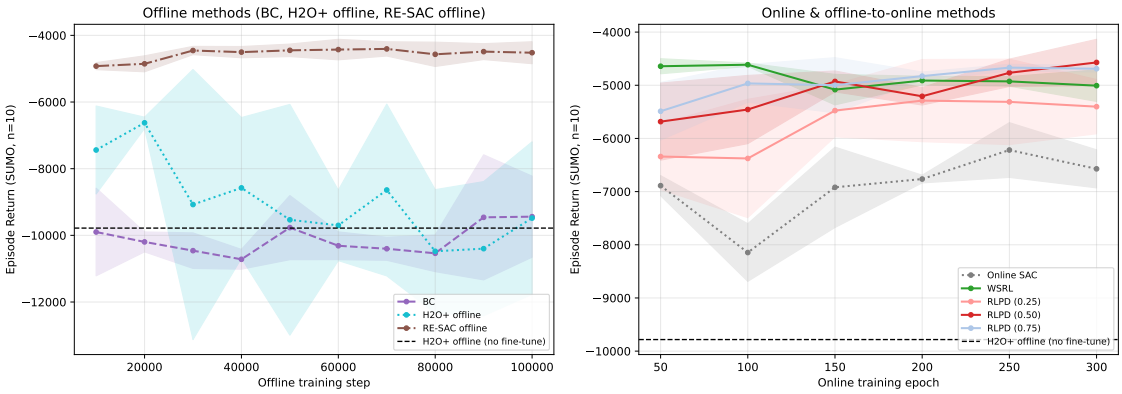

Fig. 3: Post-hoc $N = 10$ evaluation across training. Left: offline methods (step axis). RE-SAC plateaus by step 20,000 and is stable. In sharp contrast, BC and H2O+ offline both overfit: each reaches its best return early (BC at step ∼50K, H2O+ at step ∼20-30K) and then degrades steadily as training continues, ending near -9500. This is the signature pattern of offline RL without proper uncertainty regularization—extrapolation error compounds as the Q-function trains further on fixed data. Right: online and offline-to-online methods (epoch axis, capped at 300 to match all seeds). Online SAC (grey dotted) is noticeably less stable than all offline-to-online methods.

Figure 3 confirms the table-level findings. RE-SAC's curve is almost flat from step ∼20K onward, suggesting that further offline training does not help once the ensemble uncertainty has calibrated. The offline-to-online methods (WSRL, RLPD) reach their plateau around epoch 100 and remain stable. Online SAC's curve, by contrast, fluctuates by several hundred between consecutive checkpoints, indicative of value-function instability rather than lack of exploration. The left panel also makes the difference between uncertainty-regularized and unregularized offline RL stark: RE-SAC's plateau versus BC and H2O+'s continuous degradation is a direct visualisation of the value of the LCB and L_1 regularizers we introduce.

6.2 RLPD offline-ratio ablation

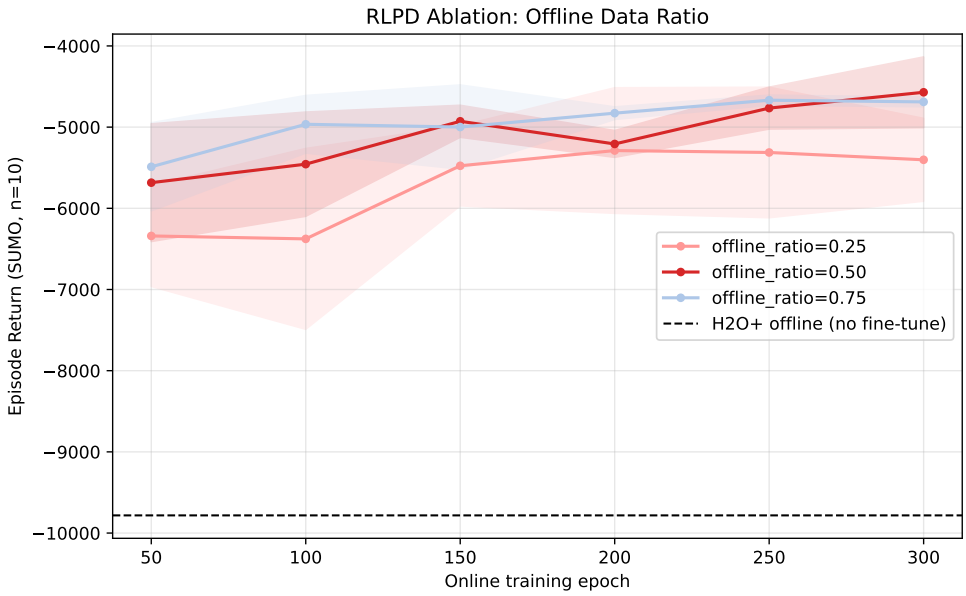

Fig. 4: RLPD offline ratio ρ. Higher ρ (more offline data per batch) accelerates early convergence; at epoch 50 the $\rho = 0.75$ variant is already $\sim$500 return ahead of $\rho = 0.25$. By epoch 300 the gap between $\rho = 0.50$ and $\rho = 0.75$ closes, while $\rho = 0.25$ lags throughout.

Figure 4 shows the ρ sweep. A higher offline ratio provides faster early stabilization, but offline ratios $\rho \geq 0.50$ converge to statistically similar endpoints. This replicates the original RLPD finding that $\rho = 0.50$ is a reasonable default.

6.3 Extended online SAC: does budget matter?

To preliminarily check whether the pure-online baseline is simply under-trained, we ran online SAC on one seed (42) for 600 epochs (double the default budget). This is a single-seed study and we report it only as an indicative observation, not as a statistical claim. The single best checkpoint in that run reached -4998 at epoch 280—close to RLPD and WSRL levels—but the policy then regressed to -6900 by epoch 300 and oscillated in the range $[-7400, -5400]$ for the remaining 300 epochs. No epoch after 280 ever recovered to the -5000 range under post-hoc $N = 10$ evaluation. This is consistent with the value-function instability visible in Fig. 3. The SAC best metric under extended training appears to be an ephemeral peak rather than a stable solution—one cannot deploy a checkpoint identified only post-hoc. A three-seed extension to confirm this trend is left for future work. In contrast, RE-SAC's best and final differ by only 253 return across three seeds (Table 2), meaning any checkpoint past step 20K is deployable.

6.4 Ablation: which regularizer matters?

To disentangle the contribution of each regularizer in RE-SAC offline, we run the algorithm with (i) no LCB pessimism ($\beta = 0$, keeping L_1 reg), (ii) no L_1 weight penalty ($\lambda_{\text{reg}} = 0$, keeping LCB $\beta = -2$), and (iii) neither regularizer (pure ensemble SAC on offline data). All ablations use the same ensemble size $E = 10$ and BC anchor, so the only thing changed is the regularizer under test. Each variant is trained for 100K steps with checkpoint saving every 10K steps, and evaluated post-hoc on $N = 10$ SUMO episodes per checkpoint across three seeds.

Three observations follow from Table 3 and Fig. 5. First, even the ensemble-only variant—which has neither LCB pessimism nor L_1 weight regularization—already attains -4303, better than every

Table 3: Ablation of RE-SAC regularizers. Same training protocol as Table 2.

Variant	Best return	Final return
Ensemble only (no LCB, no L_1)	-4303 ± 84	-4549 ± 123
L_1 only ($\beta = 0$, $\lambda = 10^{-5}$)	-4329 ± 61	-4485 ± 132
LCB only ($\beta = -2$, $\lambda = 0$)	$\mathbf{-4157 \pm 40}$	-4499 ± 31
RE-SAC offline (full: LCB + L_1)	-4267 ± 140	-4520 ± 269

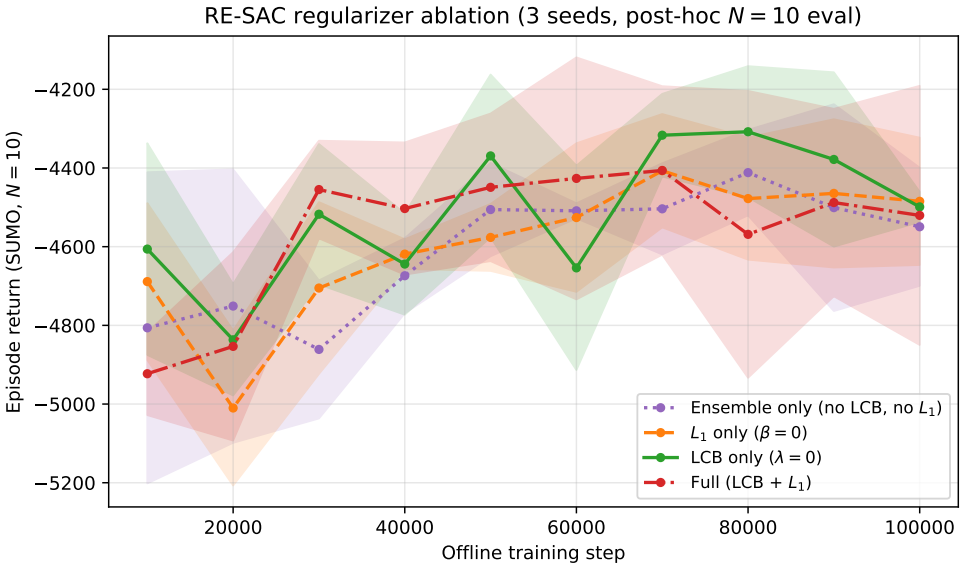

Fig. 5: Post-hoc $N = 10$ SUMO evaluation of RE-SAC ablation variants across training. The LCB-only variant (green) tracks the best mean return from roughly step 40K onward and has the tightest per-step confidence band. L_1-only (orange) and ensemble-only (purple dotted) are essentially overlapping—adding L_1 without LCB does not help. The full variant (red, LCB plus L_1) sits between LCB-only and ensemble-only with the widest band, indicating that L_1 also destabilises when added on top of LCB.

offline-to-online method in Table 2. The ensemble-Q architecture with ten heads appears to be the dominant source of RE-SAC's performance gain; the two regularizers studied here provide at most a small additional improvement on top. Second, LCB pessimism is the single most valuable ingredient: adding LCB alone yields the strongest result (-4157 ± 40), a clean 150-return improvement over ensemble-only, and also the smallest seed variance (4% of the mean). Third, the L_1 weight-norm penalty is not necessary in this offline setting: L_1 alone is statistically indistinguishable from ensemble-only (-4329 versus -4303), and combining L_1 with LCB actually performs worse than LCB alone (-4267 versus -4157) while increasing seed variance ($\sigma = 140$ versus 40). We therefore recommend the simpler ensemble-plus-LCB variant for practitioners; the full RE-SAC configuration reported in Table 2 was our initial choice for the main comparison, but the honest conclusion is that the L_1 term can be dropped without loss.

Caveat on the L_1 term. Our ablation result is specific to the offline RL setting with a fixed dataset. In prior work by the same authors on online SUMO bus control [15], a vanilla ensemble SAC variant without the L_1 penalty performed worse than with L_1, suggesting that aleatoric weight-norm regularization does help calibrate ensemble uncertainty under continuously shifting on-policy data. We thus keep L_1 in Algorithm 1 for documentation and future reuse, while noting that for pure offline training the term can be safely removed.

7 Discussion

Why does offline reinforcement learning win here? The bus-holding task has two structural properties that favor good offline methods. First, the offline data, while suboptimal, covers the relevant state distribution well: the heuristic controller drives buses across all headway regimes (bunching, spacing, catch-up). An uncertainty-aware offline algorithm can therefore extract a better policy from the dataset than the data-generating policy itself, a regime sometimes called offline policy improvement. Second, online exploration is expensive and noisy: each episode spans 18K simulator steps over a 12-line network, and the reward signal is dominated by sporadic bunching events. SAC struggles to obtain reliable Q-targets under this signal-to-noise ratio, leading to the instability in Sec. 6.3. RE-SAC sidesteps this entirely.

When would offline-to-online still pay? If the deployment environment is systematically different from the offline data distribution—for example, a new bus route or a post-construction traffic pattern—then offline-only methods cannot adapt, and the offline-to-online fine-tuning stage recovers its raison d'être. Our benchmark uses the same SUMO environment for data collection and evaluation, which is the best case for offline methods; a domain-shift benchmark is an interesting next step.

Limitations. Three seeds is modest for establishing statistical significance, though the RE-SAC-versus-RLPD gap (285 return at $\sigma \approx 300$) is near the boundary of significance; a five-seed replication would strengthen the claim. We also fix the reward scale throughout; sensitivity to reward engineering is untested. Finally, the 600-epoch SAC study was run for one seed only; a three-seed extension would be needed for a definitive conclusion on SAC's long-run behaviour.

8 Conclusion

We have shown that on a realistic SUMO bus-holding benchmark, a properly regularized pure-offline RL method (RE-SAC) outperforms two leading offline-to-online approaches (WSRL and RLPD) without any online environment interaction. Naive offline methods (BC, H2O+ offline) and pure-online SAC both fall well short. Our ablation reveals that the effective recipe is remarkably simple: an ensemble of ten Q-networks trained with LCB pessimism is sufficient; the L_1 weight-norm regularization we initially added does not help and can slightly hurt. This yields both low variance

across seeds ($\sigma = 40$ for the LCB-only variant) and stable convergence: any checkpoint past 20K offline steps is deployable. These findings suggest that practitioners working with operational transit data should consider deploying offline RL directly, without the engineering overhead of staged offline-to-online fine-tuning.

Code, data, and all evaluated checkpoints are available at https://github.com/erzhu419/offline-sumo.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 72371251), the Natural Science Foundation for Distinguished Young Scholars of Hunan Province (Grant No. 2024JJ2080), and the Key Research and Development Program of Hunan Province of China (Grant No. 2024JK2007).

References

[1] Z. Zhou, A. Peng, Q. Li, S. Levine, and A. Kumar, "Efficient online reinforcement learning fine-tuning need not retain offline data," in *International Conference on Machine Learning (ICML)*, 2024.

[2] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, "Efficient online reinforcement learning with offline data," in *International Conference on Machine Learning (ICML)*, 2023.

[3] C. F. Daganzo, "A headway-based approach to eliminate bus bunching: Systematic analysis and comparisons," *Transportation Research Part B: Methodological*, vol. 43, no. 10, pp. 913–921, 2009.

[4] J. J. Bartholdi III and D. D. Eisenstein, "A self-coordinating bus route to resist bus bunching," *Transportation Research Part B: Methodological*, vol. 46, no. 4, pp. 481–491, 2012.

[5] F. Alesiani and K. Gkiotsalitis, "Reinforcement learning for bus bunching suppression," in *IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018.

[6] J. Wang and L. Sun, "A deep reinforcement learning approach for bus holding control," *Transportation Research Part C: Emerging Technologies*, vol. 116, p. 102661, 2020.

[7] S. Levine, A. Kumar, G. Tucker, and J. Fu, "Offline reinforcement learning: Tutorial, review, and perspectives on open problems," *arXiv preprint arXiv:2005.01643*, 2020.

[8] A. Kumar, A. Zhou, G. Tucker, and S. Levine, "Conservative Q-learning for offline reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[9] I. Kostrikov, A. Nair, and S. Levine, "Offline reinforcement learning with implicit Q-learning," in *International Conference on Learning Representations (ICLR)*, 2022.

[10] X. Chen, C. Wang, Z. Zhou, and K. Ross, "Randomized ensembled double Q-learning: Learning fast without a model," in *International Conference on Learning Representations (ICLR)*, 2021.

[11] R. Yang, C. Bai, X. Ma, Z. Wang, C. Zhang, and L. Han, "RORL: Robust offline reinforcement learning via conservative smoothing," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[12] M. Nakamoto, Y. Zhai, A. Singh, M. S. Mark, Y. Ma, C. Finn, A. Kumar, and S. Levine, "Cal-QL: Calibrated offline RL pre-training for efficient online fine-tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[13] H. Niu, Y. Qiu, M. Li, G. Zhou, J. Hu, and X. Zhan, "When to trust your simulator: Dynamics-aware hybrid offline-and-online reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[14] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using SUMO," in *IEEE Intelligent Transportation Systems Conference (ITSC)*, 2018, pp. 2575–2582.

[15] Y. Zhang and L. Zheng, "RE-SAC: Disentangling aleatoric and epistemic risks in bus fleet control: A stable and robust ensemble DRL approach," 2025, in preparation.

[16] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," *Advances in Neural Information Processing Systems (NeurIPS)*, 1988.

[17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, 2018.

A Hyperparameters

Hyperparameters are split into two tables for readability: Table 4 lists values that are identical across all applicable methods, and Table 5 lists the per-method settings grouped by function (training schedule, critic architecture, offline/online mixing, and regularisers). Values were chosen following the original papers of WSRL [1], RLPD [2], and REDQ [10] with minimal tuning; the only project-specific tuning was the BC-anchor weight β_{bc} for RE-SAC, which was set to match the scale of the LCB loss.

Table 4: Common hyperparameters shared across methods.

Hyperparameter	Value
Optimizer	Adam
Learning rate	3×10^{-4}
Policy network	2 hidden layers, width 48, ReLU
Policy output	Tanh-Gaussian, $\log \sigma \in [-5, 2]$
Discount γ (SAC-family)	0.99
Soft-update coefficient τ	5×10^{-3}
Target entropy (automatic α)	$-\lvert\mathcal{A}\rvert = -2$
Random seeds	$\{42, 123, 456\}$
Data: offline buffer size	3,104,419 transitions
Data: online buffer size (SAC/WSRL/RLPD)	100,001 transitions
Episode length	18,000 simulator steps (≈ 2.5 h)

Table 5: Method-specific hyperparameters. Entries marked "—" do not apply to that method.

Hyperparameter	BC	H2O+ off.	SAC	WSRL	RLPD	RE-SAC
Training schedule						
Training length	100K steps	100K steps	300 ep	300 ep	300 ep	100K steps
Batch size	2048	2048	512	512	512	1024
Rollout batch per epoch	—	—	500	500	500	—
Warmup rollouts	—	—	500	500	500	—
UTD ratio	—	1	1	1	1	1
Checkpoint interval	10K steps	10K steps	50 ep	50 ep	50 ep	10K steps
Critic						
Q-network (width $\times$ heads)	—	48×2 twin	48×2 twin	48×2 twin	48×2 twin	48×10 ensemble
Value network (H2O+ AWR only)	—	48×1	—	—	—	—
Offline/online mixing						
Offline ratio ρ	1.0	1.0	0.0	$1.0 \to 0$	0.25/0.50/0.75	1.0
RE-SAC-specific regularisers						
LCB weight β	—	—	—	—	—	-2
OOD std weight α_{ood}	—	—	—	—	—	0.01
L_1 weight λ_{reg}	—	—	—	—	—	10^{-5}
BC anchor weight β_{bc}	—	—	—	—	—	0.005
H2O+ AWR-specific						
Expectile / quantile	—	$\tau_V = 0.7$	—	—	—	—
AWR temperature	—	$\beta_{\text{AWR}} = 3.0$	—	—	—	—
Max advantage weight	—	100	—	—	—	—

B Network architecture

All methods share a common policy/critic backbone.

Categorical embedding layer. The five categorical state features (line, bus, station, time-period, direction) are embedded separately and concatenated. Embedding table sizes: line ($12 \to 4$), bus ($389 \to 8$), station ($1 \to 2$), time-period ($1 \to 2$), direction ($2 \to 2$). Layer normalization ($\epsilon = 10^{-5}$) is applied after concatenation; a 5% dropout is applied during training. The embedding output (dimension 18) is concatenated with the 12 continuous state features to form a state vector of dimension 30.

Policy network (actor). Two hidden layers of width 48 with rectified-linear-unit activations, producing a tanh-Gaussian distribution: the mean head outputs $\mu \in \mathbb{R}^2$, the log-std head outputs $\log \sigma \in [-5, 2]$, and the action is sampled as $a = \tanh(\mu + \sigma \odot \epsilon)$ with $\epsilon \sim \mathcal{N}(0, I)$.

Q-network (critic). Two hidden layers of width 48 with rectified-linear-unit activations. For methods using a twin Q (H2O+, SAC, WSRL, RLPD), two independent Q-networks are maintained. For RE-SAC, ten ensemble heads are implemented using a vectorized-linear layer that batches all heads in one tensor operation; the forward pass outputs an $E \times B \times 1$ Q-value tensor.

Value network (H2O+ only). A single hidden layer of width 48 with rectified-linear-unit activations; H2O+ uses $V(s)$ for advantage estimation in advantage-weighted regression.

Parameter counts.

- Policy: 6,178 parameters.
- Twin Q: 7,586 parameters.
- Ensemble Q (10 heads, vectorized-linear): 37,970 parameters.
- Value network (H2O+): 3,697 parameters.
- Total embedding table: 4,012 parameters, shared across actor and critic with separate clones per network.

C Reproducibility

Seeds. All results are averaged over three seeds $\{42, 123, 456\}$. Each training script sets PyTorch and NumPy random number generator seeds at startup. The SUMO simulator seed is governed internally by the route-file stochastic demand sampling and is not set by our code; this yields genuine traffic-level stochasticity across evaluation episodes rather than deterministic replays.

Compute environment. Training was performed on a dual-GPU server ($2 \times$ NVIDIA RTX with 12 GiB each, 12 CPU cores, Ubuntu 22.04, Python 3.8, PyTorch 2.4.1 plus CUDA 12.1). Each training script fits entirely in less than 1 GiB of GPU memory, so we ran three seeds of a method concurrently per GPU. Post-hoc evaluation ran on a separate 16-core CPU workstation (no GPU was needed since SUMO is the bottleneck); a 14-worker multiprocessing pool evaluated the full set of 316 checkpoints in roughly 5 hours.

Wall-clock costs. Training wall-clock time per 100K-step run, measured as elapsed time when run three-in-parallel on one GPU:

- BC: $\sim$30 min.
- H2O+ offline: $\sim$60 min.
- RE-SAC offline: $\sim$60 min.
- Online SAC (300 epochs): $\sim$1.5 h (dominated by SUMO rollouts).
- WSRL (300 epochs): $\sim$2 h.

- RLPD (300 epochs): $\sim$2 h.

Post-hoc evaluation takes roughly 1 min per SUMO episode, so a checkpoint's $N = 10$ evaluation takes about 10 min on one CPU core.

D Data format

The offline dataset `merged_all_v2.h5` is a single HDF5 file with the following keys: `observations` ($N \times 17$, float32), `actions` ($N \times 2$, float32, in the raw tanh range $[-1, 1]$), `rewards` (N, float32), `next_observations` ($N \times 17$), `dones` (N, boolean), and `contexts` ($N \times 30$, unused in this paper but retained for compatibility with context-conditioned policies from prior work). The `BusMixedReplayBuffer` loader normalizes continuous observation dimensions to zero mean and unit variance using statistics computed once at load time and cached for evaluation.

E Code availability

Source code, the offline HDF5 dataset, all 316 evaluated checkpoints, and evaluation logs are publicly released at https://github.com/erzhu419/offline-sumo. The README provides a full reproduction recipe using a conda environment defined by `requirements.txt`; running `python auto_run.py` launches the full 22-experiment training suite with automatic GPU detection, and `python eval_checkpoints_parallel. -n_workers 14` runs the post-hoc evaluation.